

Bootcamp: Full Stack Junior

Modulo 3: Backend con PHP y Laravel

Semana 6

Contenido

1

Fundamentos de PHP para desarrolladores

2

JavaScript

3

Sintaxis avanzada de PHP

4

Programación Orientada a Objetos en PHP

5

Encapsulamiento

6

Herencia

7

Polimorfismo

Contenido:

1. Introducción a PHP

PHP es un lenguaje de programación de scripts del lado del servidor, de propósito general y de código abierto, pensado principalmente para el desarrollo de aplicaciones y sitios web dinámicos. Se integra fácilmente con HTML para generar contenido que cambia según los datos o la interacción del usuario.

1.1. Qué es exactamente PHP

- PHP significa “PHP: Hypertext Preprocessor”, un acrónimo recursivo.
- Es un lenguaje interpretado que se ejecuta en el servidor y genera HTML u otras respuestas que luego ve el navegador.
- Es software libre/código abierto y funciona en la mayoría de sistemas operativos y servidores web.

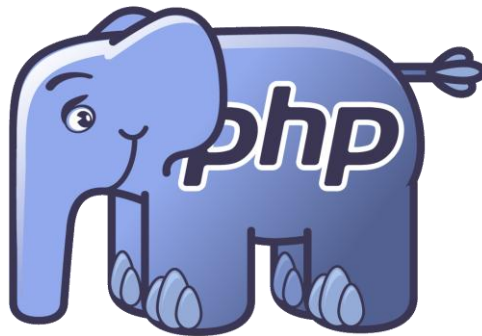


Ilustración 1 - PHP - Fuente: pngimg.com

1.2. Para qué se utiliza

Los usos más comunes de PHP incluyen:

- Sitios web dinámicos y aplicaciones web: generación de contenido que cambia según el usuario (perfiles, paneles, noticias personalizadas, etc.).
- Formularios y lógica de negocio: procesar formularios, validar datos, manejar sesiones y autenticación de usuarios (login, registro, recuperación de contraseña).

- Trabajo con bases de datos: conectarse a sistemas como MySQL o PostgreSQL para leer, escribir y actualizar datos (por ejemplo, posts de un blog, pedidos de una tienda en línea).
- CMS y e-commerce: muchos sistemas de gestión de contenidos y plataformas de comercio electrónico (como WordPress o Joomla) están escritos en PHP.
- Scripts de consola y algunas aplicaciones de escritorio (menos común, pero posible con ciertas bibliotecas).

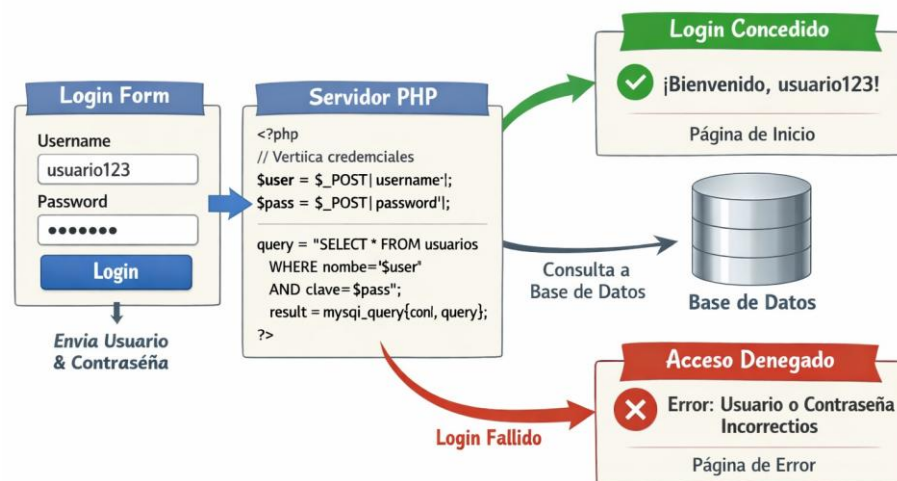


Ilustración 2 - Ejemplo típico de uso de PHP - Fuente: DALL-E

PHP funciona en un modelo cliente-servidor donde el navegador (cliente) envía solicitudes HTTP al servidor web, que procesa scripts PHP del lado del servidor y devuelve contenido dinámico como HTML. El flujo es secuencial y transparente para el usuario final, integrando PHP directamente en páginas web mediante etiquetas embebidas.

1.3. Flujo Cliente-Servidor

En este modelo, el cliente (navegador) inicia la comunicación solicitando una página vía HTTP (GET, POST, etc.).

- El navegador envía la petición al servidor web (Apache, Nginx), incluyendo URL, datos de formulario y cookies.
- El servidor web detecta extensiones .php y pasa el archivo al intérprete PHP (vía FastCGI, FPM o módulos como mod_php).
- PHP procesa el script, interactúa con bases de datos o lógica de negocio, genera HTML dinámico y lo devuelve al servidor web.

- El servidor envía la respuesta HTTP (HTML, JSON, etc.) al cliente, que la renderiza sin ver el código PHP.

Ciclo de Interacción Cliente-Servidor

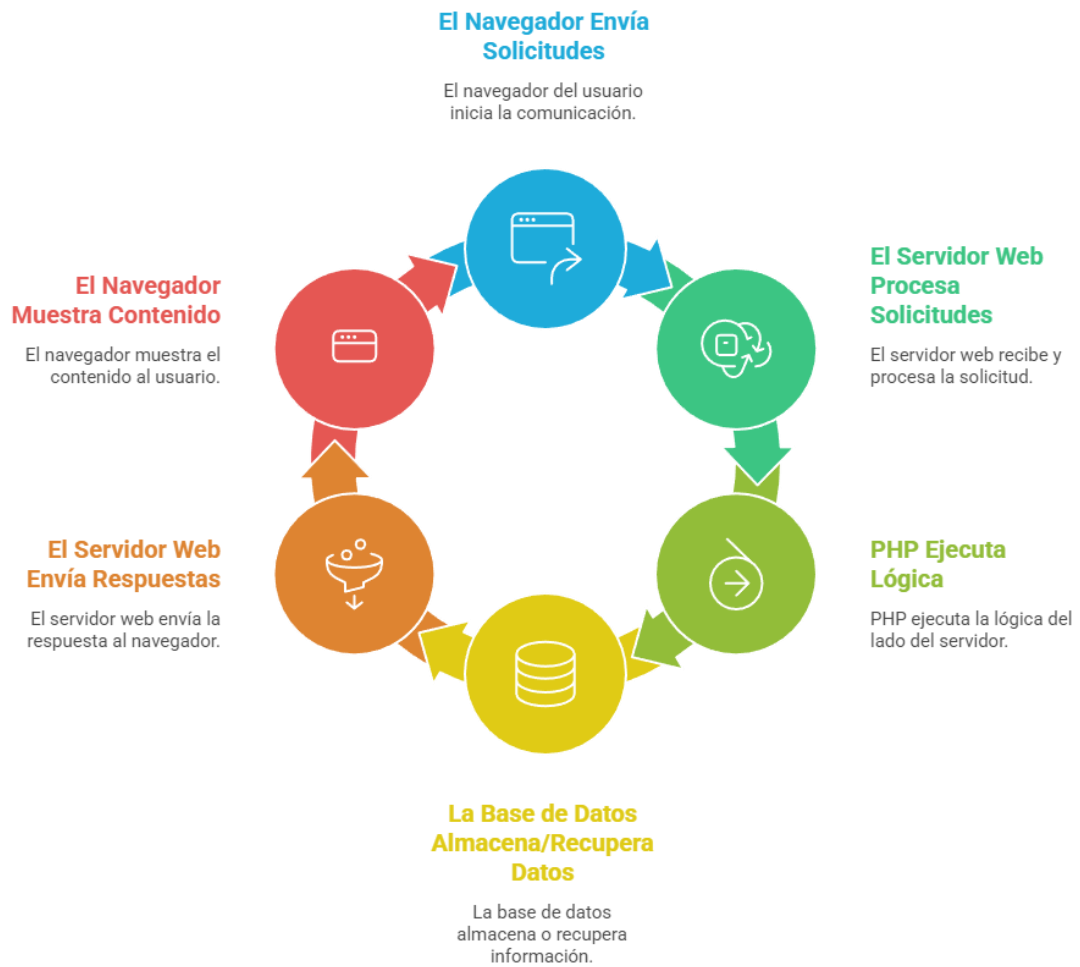


Ilustración 3 - Flujo cliente servidor - Fuente: Autor de contenidos

2. Tipado de en PHP, Estructuras de control y funciones

2.1. Tipado en PHP

PHP tiene un sistema de tipado dinámico y débil por defecto, lo que significa que las variables no requieren declaración explícita de tipo y cambian automáticamente según el valor asignado. Desde PHP 7 y especialmente en PHP 8, se han introducido mejoras como tipado estricto opcional y tipos escalares en funciones para mayor robustez.

PHP define 10 tipos de datos principales, divididos en escalares, compuestos y especiales, con un sistema de tipado dinámico:

Tipos de Datos en PHP			
CATEGORÍA	TIPO	DESCRIPCIÓN	EJEMPLO
Escalares	Integer (int)	Números enteros positivos o negativos	42, -10
	Float (double)	Números decimales o punto flotante	3.14, 2.5e3
	String	Secuencias de caracteres	"Hola", 'Mundo'
	Boolean (bool)	Valores verdadero o falso	true, false
Compuestos	Array	Colecciones indexadas o asociativas	[1, "dos"]
	Object	Instancias de clases orientadas a objetos	new Clase()
	Callable	Funciones, métodos o closures invocables	function() {}
	Iterable	Arrays u objetos traversables (PHP 7+)	foreach(\$var as \$item)
Especiales	NULL	Ausencia de valor o nulo	null
	Resource	Referencia a recursos externos (archivos, DB)	fopen("file.txt")

Ilustración 4 - Tipos de datos de PHP - Fuente: Autor de contenidos

2.2. Estructuras de control en PHP

Las estructuras de control en PHP permiten dirigir el flujo del programa mediante decisiones condicionales y repeticiones iterativas, esenciales para lógica dinámica.

Estructuras Condicionales

- Evalúan condiciones booleanas para ejecutar bloques específicos. if / else / elseif:

```
if ($edad >= 18) {
    echo "Mayor de edad";
} elseif ($edad >= 13) {
    echo "Adolescente";
} else {
    echo "Menor";
}
```

Ilustración 5 - Estructura condicional en PHP - Fuente: Autor de contenidos

Usa ?: ternario para if-else simple: `$mensaje = $aprobado ? 'Sí' : 'No';`

- switch: Multi-opción eficiente vs múltiples elseif.

```
switch ($dia) {  
    case 'lunes':  
        echo 'Inicio semana';  
        break;  
    case 'viernes':  
        echo 'Fin semana';  
        break;  
    default:  
        echo 'Día normal';  
}
```

Ilustración 6 - Estructura switch en PHP - Fuente: Autor de contenidos

Bucles Iterativos

Repiten código mientras se cumpla una condición.

Bucles en PHP		
Estructuras de Control Repetitivas		
BUCLE	SINTAXIS BÁSICA	CUÁNDO USAR
 while	<pre>while (\$i < 10) { \$i++; }</pre>	Condición desconocida al inicio
 do-while	<pre>do { \$i++; } while (\$i < 10);</pre>	Ejecutar al menos una vez
 for	<pre>for (\$i=0; \$i<10; \$i++) { ... }</pre>	Número de iteraciones conocido
 foreach	<pre>foreach (\$array as \$item) { ... }</pre>	Recorrer arrays/objetos

Ilustración 7 - Bucles en PHP - Fuente: Autor de contenidos

2.3. Funciones en PHP

Las funciones en PHP son bloques de código reutilizables que encapsulan lógica, reciben parámetros opcionales y devuelven valores mediante return, promoviendo modularidad y mantenibilidad.

```
function nombreFuncion($parametro1, $parametro2 = "default") {
    // Lógica de la función
    $resultado = $parametro1 * 2;
    return $resultado; // Opcional
}

// Invocación
echo nombreFuncion(5); // 10
```

Ilustración 8 – Funciones en PHP - Fuente: Autor de contenidos

Tipos de Parámetros

⚙️ Parámetros de Funciones en PHP		
Tipos de Argumentos y Parámetros		
TIPO	SINTAXIS	DESCRIPCIÓN
Posicionales	<code>function suma(\$a, \$b)</code>	Orden fijo
Por defecto	<code>function greet(\$name = "Invitado")</code>	Valores opcionales al final
Referencia	<code>function incr(&\$num)</code>	Modifica variable original &
Tipados PHP7+	<code>function div(int \$a, int \$b): float</code>	Validación estricta
Variádicos	<code>function lista(...\$nums)</code>	Argumentos múltiples como array

Ilustración 9 – Tipos de parámetros en PHP - Fuente: Autor de contenidos

3. Manejo de errores en PHP

3.1. Errores Tradicionales vs Excepciones

PHP maneja errores principalmente mediante excepciones con bloques try-catch-finally (desde PHP 5+), configuraciones globales como error_reporting() y funciones como set_error_handler() para errores tradicionales.

3.2. Bloques Try-Catch-Finally

Esta estructura captura excepciones lanzadas con throw, permitiendo un flujo controlado sin detener el script abruptamente.


```

try {
    // Código riesgoso (ej: consulta DB, archivo)
    $resultado = dividir(10, 0);
} catch (Exception $e) {
    // Manejo específico: log, mensaje usuario
    echo "Error: " . $e->getMessage();
} catch (DivisionByZeroError $e) {
    // Múltiples catch para tipos específicos (orden importa)
    echo "División inválida";
} finally {
    // Siempre se ejecuta (limpieza: cerrar conexiones)
    echo "Recursos liberados";
}

```

Ilustración 10 - Manejo de errores en PHP - Fuente: Autor de contenidos

3.3. Errores Tradicionales vs Excepciones

PHP distingue errores fatales/recuperables de excepciones (Throwable desde PHP 7).

Tipo	Manejo principal	Ejemplo
Errores	error_reporting(), set_error_handler()	División por cero, memoria agotada
Excepciones	try-catch	throw new Exception("Msg")
Custom	Extender Exception	class ValidacionError extends Exception {}

Tabla 1 - Errores y Excepciones - Fuente: Autor de contenidos

4. Introducción a la POO

La Programación Orientada a Objetos (POO) en PHP organiza el código en clases y objetos para modelar entidades del mundo real, promoviendo reutilización, modularidad y mantenimiento.

La POO se basa en clases (plantillas) y objetos (instancias concretas) con propiedades (datos) y métodos (funciones).

```

class Coche {
    public $marca;      // Propiedad pública
    public $modelo;

    public function __construct($marca, $modelo) {
        $this->marca = $marca; // $this referencia el objeto actual
        $this->modelo = $modelo;
    }

    public function mostrarInfo() {
        return "Coche: $this->marca $this->modelo";
    }
}

$miCoche = new Coche("Toyota", "Corolla"); // Crear objeto
echo $miCoche->mostrarInfo();[web:45][web:49]

```

Ilustración 11 - POO en PHP - Fuente: Autor de contenidos

4.1. Pilares clave de POO

Pilares de la Programación Orientada a Objetos		
PILAR	DESCRIPCIÓN	EJEMPLO
• Encapsulación	Oculta datos internos; usa <code>public</code> , <code>private</code> , <code>protected</code> getter/setter	<code>private \$password;</code>
• Herencia	Clase hija extiende clase padre con <code>extends</code>	<code>class CocheElectrico extends Coche</code>
• Polimorfismo	Mismo método se comporta diferente por clase. Sobrescribir <code>arrancar()</code> en subclase	<code>public function arrancar() { ... }</code>
• Abstracción	Clases abstractas/interfaces definen contratos	<code>abstract class Vehiculo {</code> <code>abstract function mover();</code> <code>}</code>

Ilustración 12 - Pilares de la POO - Fuente: Autor de contenidos

PHP soporta POO completa desde PHP 5 (mejorada en PHP 7/8 con traits, tipos escalares, etc.).

5. PHP en la actualidad

PHP sigue siendo el lenguaje que impulsa ~75-77% de sitios web globales, con PHP 8.4 como versión estable principal y PHP 9 en desarrollo prometiendo revitalización.

PHP mantiene un ecosistema maduro y activo, con soporte oficial hasta 2028+ para ramas recientes (8.2-8.4), aunque versiones antiguas como 8.1 ya son EOL y requieren migración para seguridad. Frameworks como Laravel y Symfony lideran adopción, ofreciendo rendimiento JIT mejorado, POO avanzada y compatibilidad cloud/microservicios.

Fortalezas y Retos

- Fortalezas: Comunidad masiva (WordPress 40% web), entrega rápida, bajo costo hosting, evolución constante (JIT, enums, atributos PHP 8+).
- Retos: Legado obsoleto en muchas apps requiere refactor (50-60% código reescrito para PHP 8+), competencia en real-time de Node/Go.
- Tendencia: Creciente uso en SaaS robustos y APIs; IA acelera desarrollo, pero estabilidad PHP gana en legacy/maintenance.

PHP no está "muerto", sino evolucionando como opción pragmática y confiable para web tradicional/escalable

Recursos de Apoyo



Material de Lectura

[HP Manual \(español\): Documentación completa con ejemplos interactivos. Ideal para referencias rápidas.](#)

...



Material de Lectura

[PHP The Right Way: Mejores prácticas modernas \(PSR, seguridad, rendimiento\).](#)



Material de Lectura

[Manejo Excepciones Try-Catch: Tutorial práctico con código.](#)



Material de Lectura

[PHP.net Supported Versions: Estado versiones y migraciones.](#)